

Nome: _____ Turma: _____

Atenção: Este teste tem 7 questões em 4 páginas.

Responda directamente no enunciado. Fundamente todas as respostas.

1. Pretende-se que o seguinte fragmento de código *assembly* MIPS conte o número de elementos de um vector de inteiros que são menores que um dado valor (limiar) e guarde o resultado em `$s1`. O vector contém pelo menos um elemento.

Assuma a seguinte atribuição de registos:

`$s7`: base do vector `$s0`: número de elementos do vector `$s5`: valor de limiarO resultado é guardado em `$s1`.

- [2,5] (a) Complete o código.

```
1)          addi  $s1, $zero, 0
2) Ciclo: lw   $t0, 0($s7)
3)          slt   $t1, $t0, $s5
4)          beq   $t1, $zero, Cont
5)          addi  $s1, $s1, 1
6) Cont:     addi  $s7, $s7, 4
7)          addi  $s0, $s0, -1
8)          bne   $s0, $zero, Ciclo
```

- [1] (b) Pretende-se alterar o fragmento para contabilizar os elementos situados *entre* dois valores (valores iguais aos limiares não devem ser contabilizados). O limiar superior está em `$s5` (como anteriormente) e o inferior em `$s4`.

Acrescente instruções entre as linhas 4 e 5 de forma a obter o efeito pretendido.

```
...
4)          beq   $t1, $zero, Cont
            slt   $t2, $s4, $t0          #$t2 ou qualquer registo livre
            beq   $t2, $zero, Cont
5)          addi  $s1, $s1, 1
            ...
```

2. Considere a implementação da unidade de processamento uniciclo apresentada em anexo.

- [2] (a) Durante a execução de uma dada instrução, os valores de alguns sinais internos são os seguintes:

`MemtoReg=1` e `RegWrite=1`Identifique a instrução e indique o valor do sinal interno `ALUSrc`.

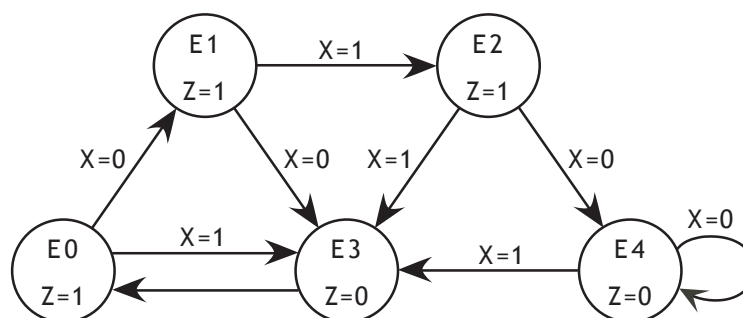
Trata-se de instrução `lw`, a única que carrega um registo com um valor obtido da memória de dados.

Na execução da instrução `lw`, tem-se `ALUSrc=1`, porque o segundo operando usado no cálculo de endereços (operação efectuada na ALU) é o valor imediato incluído na instrução.

- [1] (b) Num circuito integrado com a unidade de processamento defeituosa, o sinal `PCSrc` está sempre a zero. Que instruções são afectadas e como?

A única instrução durante cuja execução temos `PCSrc=1` é a instrução de salto condicional relativo (`beq`). Logo, apenas esta é afectada pelo defeito de fabrico. Na presença do defeito temos sempre `PCSrc=0`, pelo que os saltos condicionais nunca são tomados, mesmo quando a condição é verdadeira.

- [2] 3. A figura seguinte apresenta o diagrama de estados de um circuito sequencial com uma entrada X e uma saída Z .



Admita E2 como estado inicial e que nos seis primeiros ciclos de relógio os valores de X são, respectivamente, 0, 1, 1, 0, 0 e 0. Determine a sequência de estados e de valores na saída Z .

Estados: E4, E3, E0, E1, E3, E0
Saída Z: 0; 0; 1; 1; 0; 1

4. Considere o seguinte fragmento de código *assembly* a ser executado num processador MIPS com *pipeline* e atalhos. O processador suporta prognóstico de saltos condicionais e avalia os saltos condicionais no andar ID.

```

L:    lw    $s2, 0($s3)
      add   $t0, $s2, $t0
      andi  $t6, $s2, 0x0001
      add   $t2, $t2, $t6
      sub   $t1, $t1, $s2
      addi  $s3, $s3, 4
  
```

- [2] (a) Indique todas as dependências directas de dados existentes no fragmento.

1. `add` (linha 2) depende de `lw` via `$s2`
2. `andi` depende de `lw` via `$s2`
3. `add` (linha 4) depende de `andi` via `$t6`
4. `sub` depende de `lw` via `$s2`

- [1,5] (b) Para *cada* dependência, indique se provoca um conflito e, em caso afirmativo, como é que ele é resolvido: por protelamento e/ou por utilização de atalhos. Especifique claramente todos os atalhos envolvidos.

1. causa protelamento; usa atalho MEM→ALU (entrada superior);
2. sem protelamento; não necessita de atalho;
3. sem protelamento; usa atalho ALU→ALU (entrada inferior);
4. sem protelamento; não necessita de atalho.

- [1] (c) Determine o CPI deste fragmento.

Número de ciclos: 11; número de instruções: 6

$$\text{CPI} = 11/6$$

- [1] (d) O fragmento de código é incluído num ciclo, conforme se indica a seguir:

```

L:      lw      $s2, 0($s3)
        add     $t0, $s2, $t0
        andi    $t6, $s2, 0x0001
        add     $t2, $t2, $t6
        sub     $t1, $t1, $s2
        addi    $s3, $s3, 4
        bne     $s3, $s0, L

```

Sabendo que este novo fragmento efectua três iterações (i.e., na terceira execução da instrução `bne` o salto não é tomado), determine o seu novo CPI.

Considerar um protelamento como equivalente a uma instrução `nop`.

Existe um protelamento adicional sempre que o salto `bne` é tomado (2 protelamentos por iteração, *excepto na última*).

Número equivalente de instruções: $9 \times 2 + 8 = 26$

Número de ciclos: $4 + \text{número de instruções} = 30$

$$\text{CPI} = 30 / 21 = 10 / 7 \approx 1,43$$

5. Um processador funciona a 2 GHz e possui uma memória *cache* unificada. A taxa de falhas no acesso à cache é 9% e o tempo de acesso à memória principal é 50 ns.

[1] (a) Calcule a penalidade de falha.

$$T = \frac{1}{F} = 0,5 \text{ ns} \quad \text{logo} \quad p_f = \frac{50}{0,5} = 100$$

[2] (b) Medições efectuadas para um conjunto de *benchmarks* revelaram que o número de ciclos de protelamento em acessos a memória (por instrução) é $\text{CPI}_{\text{prot}} = 12$.

Determine a percentagem de instruções que acede a dados.

n_a : nº de acessos a memória por instrução
 t_f : taxa de falhas

$$\text{CPI}_{\text{prot}} = t_f \times p_f \times n_a \quad \rightarrow \quad n_a = \frac{12}{0,09 \times 100} = \frac{4}{3}$$

Como cada instrução implica um acesso para a sua obtenção, os restantes são para acesso a dados: $n_a - 1 = \frac{1}{3} = 33,3\%$

- [2] 6. Um CPU usa endereçamento por bytes. A sua memória cache de instruções é acedida por palavra e tem as seguintes características:
- bits de endereço: 24
 - número de palavras/bloco: 8
 - número de vias: 4
 - número de conjuntos: 64

Determine o tamanho (em bits) da etiqueta, do índice e do deslocamento de palavra no bloco.

Deslocamento: $8 = 2^3$ palavras/bloco $\rightarrow$ 3 bits

Índice: $64 = 2^6$ conjuntos $\rightarrow$ 6 bits

Etiqueta: Não considerar os 2 bits menos significativos do endereço (endereços de palavra têm sempre esses bits a zero):

$$22 - 3 - 6 = 22 - 9 = 13 \text{ bits}$$

- [1] 7. Tipicamente, a taxa de falhas de uma memória cache diminui à medida que aumenta o tamanho dos blocos. Contudo, a partir de uma certa dimensão dos blocos, a taxa de falhas aumenta. Explique este comportamento, indicando a(s) causa(s).

O aumento do tamanho do bloco permite aproveitar melhor a localidade (proximidade) espacial de instruções ou dados. Mas para blocos cada vez maiores, cresce a probabilidade de algumas partes deles não serem usadas. Portanto, a capacidade útil da cache é reduzida e, consequentemente, a taxa de falhas aumentará.